

Problem 1: Loop Invariant

- (a) The loop invariant of `mystery5(a,n)` should be $r \cdot b^i = a^n$. There are two cases in the loop for i is odd and i is even.
- (i) Case 1: if $(i \% 2 == 0)$, or if i is even. $b = b * b$ or b^2 and $i = i / 2$. Since the base b is squared and exponent i is halved, as $(b^2) \cdot (i / 2)$ still equals to b^i , and r remains to be 1, the invariant holds true before and after the loop.
 - (ii) Case 2: if i is odd. $r = r * b$ and $i = i - 1$. r in this case will take one factor of b and $i = i - 1$ accordingly, as $(r \cdot b) \cdot b^{(i-1)}$ still equals to $r \cdot b^i$. So $r \cdot b^i$ still remains no change after the loop proving the invariant is true.

Therefore, the loop invariant for this code is $r \cdot b^i = a^n$.

- (b) Initial state: $r = 1, b = a, i = n$

$$\text{LHS} = r \cdot b^i = (1) \cdot (a)^n = a^n = \text{RHS}$$

So the initial state is true before the loop start.

- (c) For two cases: i is even and i is odd

- (i) Case 1: if $(i \% 2 == 0)$, or if i is even.
 - (1) Execution: $b = b * 2, i = i / 2$
 - (2) Initial state: $r \cdot b^i = a^n$
 - (3) After state: $r \cdot (b^2)^{(i/2)} = r \cdot b^i = a^n$

Therefore, the invariant holds true after i is even loop.

- (ii) Case 2: if i is odd
 - (1) Execution: $r = r \cdot b, i = i - 1$
 - (2) Initial state: $r \cdot b^i = a^n$
 - (3) After state: $(r \cdot b) \cdot b^{(i-1)} = r \cdot b^i = a^n$

Therefore, the invariant holds true after i is odd loop.

Since the invariant holds true after two separate cases of loop, the invariant is true.

- (d) The second to last loop, where i eventually become 2 which is the last case for i is even, it will be divided by 2 again becoming 1. The last loop for $i > 0$ as $i = 1$, it goes to i is odd case and i eventually turns to zero. At the same time, r will record the last b value which after all of the iterations and following the loop invariant proven above, should be b^i or a^n at the end. So for the situation after the loop: $r \cdot b^0 = r \cdot 1 = a^n$. Since r records the last b value after all of the iterations and combines with the number of exponents taken by r for odd cases, as $b^{(i - \text{even})} \cdot b^{(i - \text{odd})} = b^{(2i - (\text{even} + \text{odd}))} = b^{(2i - i)}$ returns back to b^i . So at the end $r = b^i = a^n$. So returning r will return a^n .
- (e) The loop will repeatedly go to the even case since $i - 1$ will be operated if i becomes odd anytime which will make i becomes an even number. So i will end up as 2 for the last even loop and i becomes 1 after the last even case so it will go to the odd case making $i = i - 1$ so $1 - 1 = 0$. Since the loop will run when $i > 0$, when it eventually becomes 0, the loop will stop.

For even case, since i is reduced by a factor of 2, it will run at $O(\lg n)$ and $\Omega(\lg n)$ time. For the odd case, $i - 1$ operation is done for exactly the number of 1's in the binary representation of the number i . Any positive integer has less or equal to $\lg(n)$ bits, so in the odd case the worst runtime is $O(\lg n)$. The best case in the odd case is $\Omega(1)$ when the most significant bit of the number is 1 and rest are 0's. For the worst case, $O(\lg n) + O(\lg n) = O(\lg n)$. For the best case, $\Omega(\lg n) + \Omega(1) = \Omega(\lg n)$. Since the worst case runtime is $O(\lg n)$ and the best case runtime is $\Omega(\lg n)$, the runtime for `mystery5` is $\Theta(\lg n)$.

Problem 2: Ranking Functions

$1/n$
 42
 $\lg \lg n$
 $\lg(n \lg n), \lg(n^2)$
 $\lg(n)^2$
 $n^{(1/3)}$
 $\sqrt{n}, \sqrt{2}^{\lg n}$
 n
 $\log_{10}(n^n), \lg(n!)$
 $n^2, 9^{\log_3(n)}$
 $4^{\sqrt{n}}$
 $(3/2)^n$
 2^n
 2^{2n}
 $n!$
 n^n

Problem 3: Prove or Disprove

- (a) Let $f(n) = 2n$, $g(n) = n$.

By definition of Big-O notation, $2n \leq c \cdot n$ so $f(n) = O(g(n))$.

However, $2^{f(n)} = 2^{2n} = 4^n$, while $2^{g(n)} = 2^n$. For any random c , $4^n \not\leq c \cdot 2^n$, so $2^{f(n)} \neq O(2^{g(n)})$. Therefore, the statement is false.

- (b) Define $f(n)$ to be n^2 when n is even and $(n+4)^2$ when n is odd, we need to prove $f(n) = \Theta(n^2)$ first. For even cases, there exists positive constants c_1, c_2 and n_0 such that $0 < c_1 \cdot n^2 \leq n^2 \leq c_2 \cdot n^2$ for all $n \geq n_0$. For odd cases, there exists positive constants c_1, c_2 and n_0 such that $0 < c_1 \cdot n^2 \leq (n+4)^2 \leq c_2 \cdot n^2$ for all $n \geq n_0$ holds true as well since $(n+4)^2$ is simply $n^2 + 8n + 16$ with a leading term n^2 as its rate of growth. This proves that $f(n) = \Theta(n^2)$. Then moving on to proving non-decreasing, while it is true for all even cases, $f(n) \leq f(n+1)$, for the odd cases $f(n)$ is in fact greater than $f(n+1)$ since $(n+4)^2 > (n+1)^2$. Therefore, f is not asymptotically non-decreasing for some cases of $f(n)$ such as the one defined above. Thus, the statement is false.
- (c) Since $f(n)$ is asymptotically positive such that $f(n) > 0$ for sufficiently large n and asymptotically non-decreasing such that for all $n \geq n_0$, $f(n) \leq f(n+1)$, we know that for any $n \geq n_0$ and a positive constant c , $f(n) \leq c \cdot f^2(n)$ since $f^2(n) = f(n) \cdot f(n)$. Asymptotically positive and non-decreasing together makes $f(n)$ being positive and greater for any n values, and this makes sure that the square of $f(n)$ at any n value is greater than $f(n)$. Therefore, $f(n)$ is $O(f^2(n))$. Thus, the statement is true.

Problem 4: Merge-Sort Array Space

- (a) For the first divide it creates two new arrays L and R that each takes up $n/2 + 1$ space and in total $n + 2$ space. For the second divide it creates four new arrays that each takes up $n/4 + 1$ space and in total $n + 4$ or $2 \cdot (n/2 + 2)$. So on we can find the pattern that for i levels down into the division the space for new arrays defined at each level is $2^i \cdot (n/2^i + 2) = n + 2 \cdot 2^i$. And for merge sort we know that there are total of $\lg n$ levels of divide. To find the total space that all new arrays used together at the end we will do a geometric sum from $i = 0$ to $\lg n - 1$ of $n + 2 \cdot 2^i$ which is equal to $n \lg n + 2 \cdot \sum_{i=0}^{\lg n - 1} 2^i$. Sum from $i = 0$ to $\lg n - 1$ of 2^i can be simplified to $2^{\lg n} - 1$ and this simplifies to $n - 1$. So the formula becomes $n \lg n + 2(n - 1)$ which is $n \lg n + (2n - 2)$. Since $n \lg n$ is the leading term and grows exceptionally faster than n or 2 as constant, it is the rate of growth for the function. Thus, $A(n) = \Theta(n \lg n)$.

- (b) If we now can recycle memory after each merge is done, then the array space is determined by the largest array that will be used at any time during the sort, and the small space for allocation and constant can be ignored. This is where we finish our final merge into the main array as we are holding two $n/2 + 1$ arrays to do the final merge. So $S(n)$ is simply $\Theta(2 \cdot (n/2)) + \Theta(1)$ which is $\Theta(n)$.

Problem 5: Find the Missing Integer

- (a) For this problem, the intuition would be counting the number of ones and zeros at every bit for a turn. Starting at the most significant bit, if there is a missing number, then there must be at least one missing one or zero at the significant bit due to the nature of binary numbers. Moving on to the next bit, it will continue to have one missing one or zero. Always focus on the smaller group and eventually the missing number will be sorted out. Throughout the process, we will be keeping an int x for the final answer, whenever there is a missing one, x will be added by 2 to the power of current bit. We will also have the smaller group either one or zero and their original index from A being kept as an array to the input of the function. Finally, we will have a index keeper to move to the next most significant bit after every round of check. Here is the code:

```
def find_missing(k, n, less, x, index):
    if n == 0:
        return 0
    if k < 0:
        return x
    ones = [ ]
    zeros = [ ]
    for i in less:
        if get(i, index) == 0:
            zeros.append(i)
        else:
            ones.append(i)
    if len(zeros) < len(ones):
        return find_missing(k - 1, len(zeros), zeros, x, index + 1)
    else:
        x += int(math.pow(2, k))
        return find_missing(k - 1, len(ones), ones, x, index + 1)

missing_number = find_missing(k - 1, n, [num for num in range(0,n)], 0, 0)
```

- (b) The worst case for this algorithm is that there is only one missing number in A . If this is the case, then every time the smaller group will only have one number less than the group with more zeros or ones. For the first scan, it will take $O(n)$ time. For the second scan, since we separated zeros and ones into two even group with one group having one number less, it would take $O(n/2)$ time to scan it. Going down the level it will halve the scan time every time so the runtime looks like $O(n) + O(n/2) + O(n/4) \dots$ and for the number of times we need to perform the scan until the group size reduce to only one number will be $O(\lg n)$ times because as we are scanning the binary bits, for an array having 2^k integers, the maximum bits the number can have is k . For the geometric sum from $k = 1$ to $\lg n - 1$ of $n / (2^k)$, it would be $n \cdot (1 / 1 - 1/2)$ which would be $2n$ which is $O(n)$. So the worst case runtime for this algorithm is $O(n)$.